

Assignments:

1. Use git tag to create and manage tags.

Ans: In Git, a tag is a reference to a specific commit in a repository. Tags are used to mark important points in the history of a project, such as releases or versions.

The git tag command is used to manage tags in a Git repository. Here are some common uses of the command:

- a. `git tag` → lists all the tags in the repository.
- b. `git tag -a <tag_name> -m "<tag_message>"` → creates a new tag with a message.
- c. `git tag -d <tag_name>` → deletes a tag.
- d. `git push origin --tags` → pushes all tags to the remote repository.
- e. `git checkout <tag_name>` → checks out the code at the specified tag. Means HEAD is changed to that particular tag commit.
- f. `git show <tag_name>` → See Tag Details
- g. `git restore --source=<tag_name> <file-name>` → To restore a file from that tag
- h. `git restore --source=<tag_name> .` → Restore Multiple Files

2. Compare git revert vs git reset vs git restore.

Ans:

git revert:

- Reverts a specific commit, creating a new commit that reverses the changes.
 - Preserves the original commit history.
 - Useful for undoing changes while maintaining a clean history.
- Example: `git revert <commit_hash>`

git reset:

- Resets the current branch to a specific commit, discarding all changes after that commit.
 - Options:
 - `--soft`: Resets the branch pointer, but keeps changes in the staging area.
 - `--hard`: Resets the branch pointer and discards all changes.
 - Alters the commit history.
 - Useful for undoing changes and starting fresh from a specific point.
- Example: `git reset <commit_hash>` (or `--soft` or `--hard`)

git restore:

- Restores files or directories to a specific commit or state.
- Can restore files from a specific commit, index, or the current working tree.
- Does not create a new commit.

- Useful for recovering files or undoing changes to specific files.

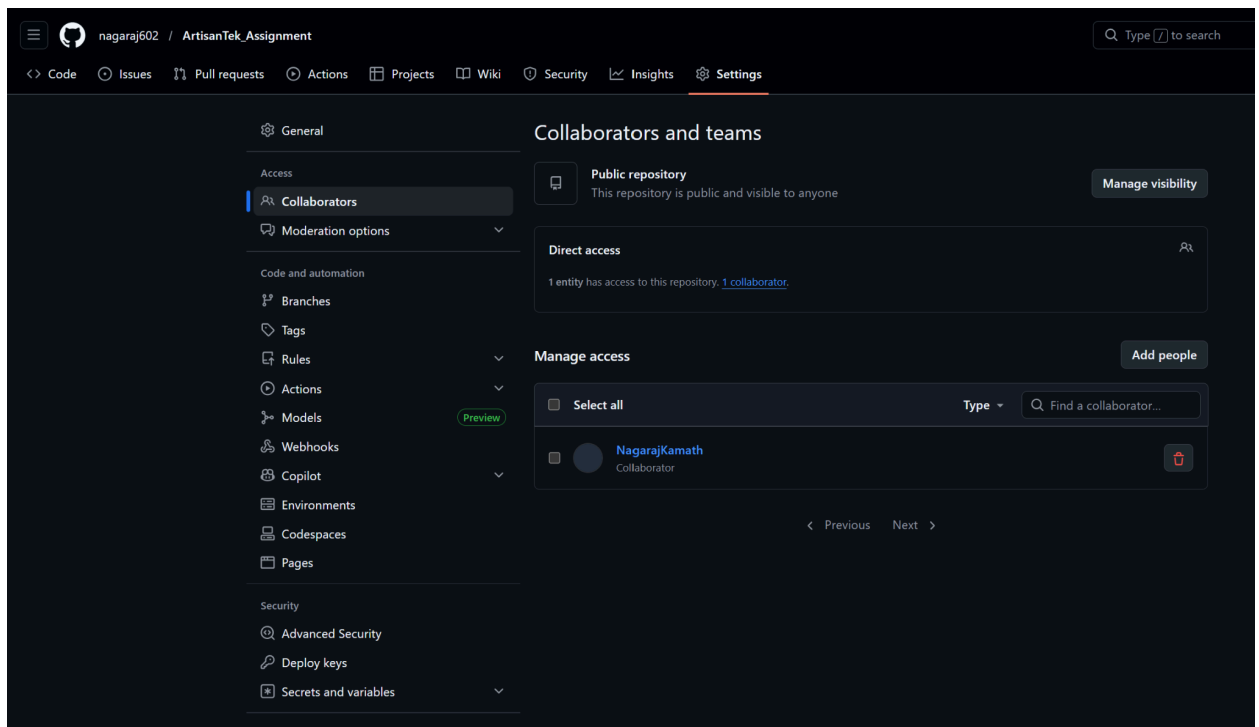
3. Create a pull request, add collaborators, and get reviews.

Ans:

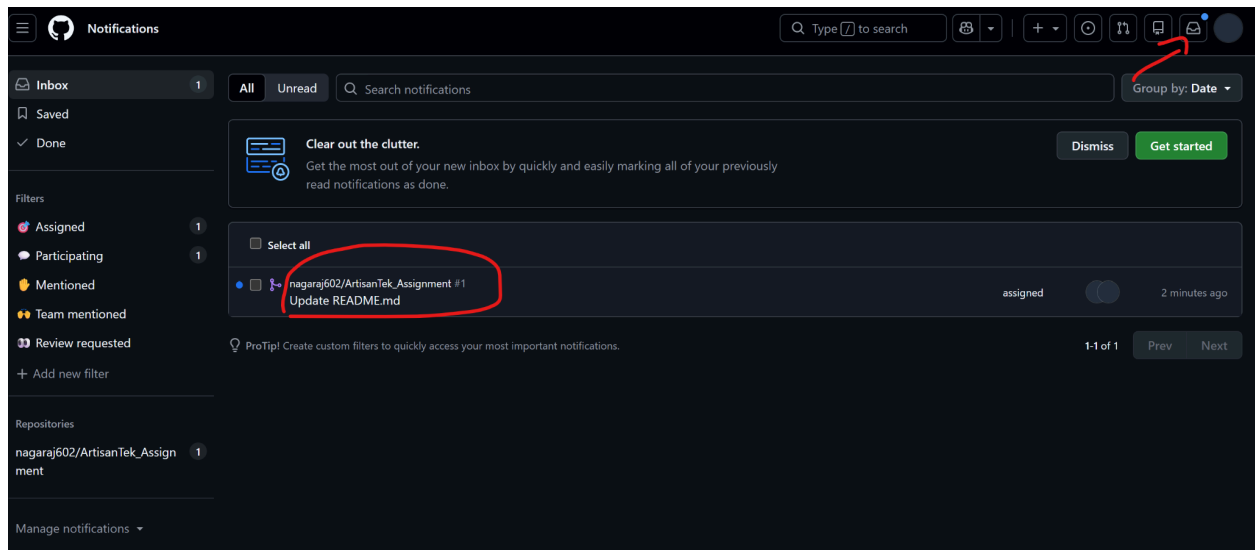
Login to Github >> Create a repo >> Go to settings >> collaborators >> Click on Add people >> Add the username or email address of the collaborator we are inviting: NagarajKamath >> The collaborator needs to accept the invitation from their end.

Now I created a branch named: test and made changes in the README.md file. Then we will find a compare and pull request or else create a pull request and add a reviewer and also add an assignee (who has authority to merge the pull request).

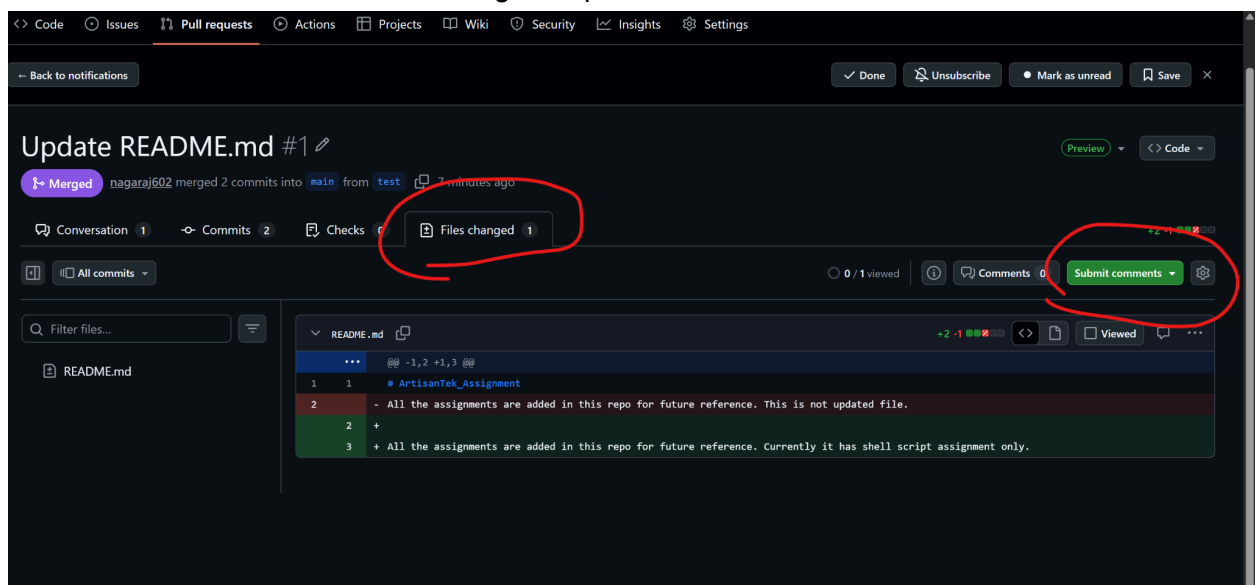
The collaborator will check the changes and approve requests by clicking on File changed >> submit review. If the reviewer approves the changes, then we can merge the branch. If the reviewer asks to make some changes, then we need to update the changes and send for approval again. After getting approval, we will merge the branch.



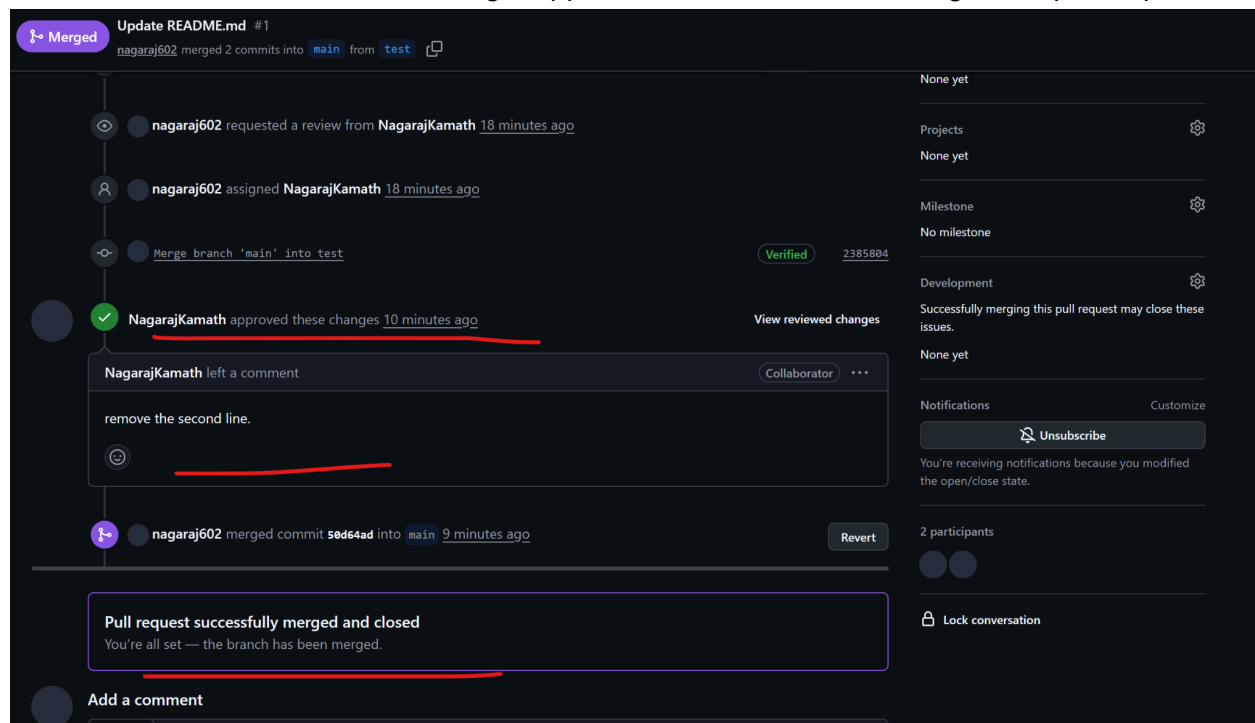
Screenshot from collaborator account, after accessing the invitation:



Screenshot from reviewer end showing the option of submit review:



Screenshot from the owner end who got approval from reviewer and merged the pull request:



4. Understand GitHub Actions for CI/CD. --> this can be ignored for now

5. Task1: Branching & Merging

- Initialize a new Git repo called git_practice.
- Create a new branch feature/login.
- Add a file login.sh and commit changes.
- Switch to main, create another file readme.md, and commit.
- Merge feature/login into main.
- Resolve any merge conflicts if they occur.
- Push all branches to GitHub.

Ans:

```
→ mkdir git_practice
→ cd git_practice
→ git init
→ touch README.md
→ git add .
→ git commit -m "Added README.md file"
→ git branch
→ git checkout -b login
→ touch login.sh
→ git add .
→ git commit -m "Added login.sh file in login branch"
→ git switch main
```

→ touch readme.md
→ vi README.md
Added one line in README.md
→ git add .
→ git commit -m "Added a line README.md"
→ git switch login
→ vi README.md
Added different text in same line
→ git add .
→ git commit -m "Added different line in README.md file"
→ git switch main
→ git merge login

o/p: Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge **failed**; fix conflicts and then commit the result.
→ vi README.md
Resolved the merge conflict.
→ git add .
→ git commit -m "Resolved merge conflicts"

Now we push to remote repo by linking remote repo URL to local repo. First create an empty repo without README.md file in github.com.

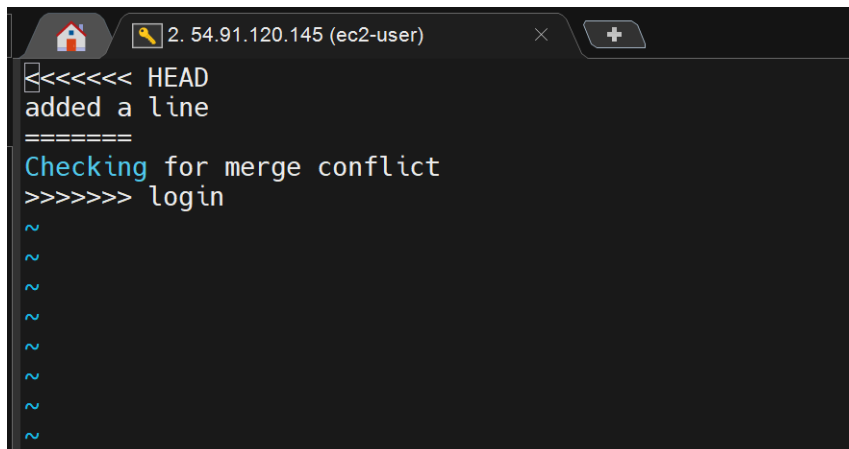
→ git remote add origin https://github.com/nagaraj602/git_practice.git
→ git push --set-upstream origin main
→ git push --set-upstream origin login

```
[ec2-user@ip-172-31-23-178 ~]$ mkdir git_practice
[ec2-user@ip-172-31-23-178 ~]$ cd git_practice
[ec2-user@ip-172-31-23-178 git_practice]$ git init
Initialized empty Git repository in /home/ec2-user/git_practice/.git/
[ec2-user@ip-172-31-23-178 git_practice]$ touch README.md
[ec2-user@ip-172-31-23-178 git_practice]$ git add .
[ec2-user@ip-172-31-23-178 git_practice]$ git commit -m "Added README.md file"
[main (root-commit) 56c76de] Added README.md file
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md
[ec2-user@ip-172-31-23-178 git_practice]$ git branch
* main
[ec2-user@ip-172-31-23-178 git_practice]$ git checkout -b login
Switched to a new branch 'login'
[ec2-user@ip-172-31-23-178 git_practice]$ git branch
* login
  main
[ec2-user@ip-172-31-23-178 git_practice]$ touch login.sh
[ec2-user@ip-172-31-23-178 git_practice]$ git add .
[ec2-user@ip-172-31-23-178 git_practice]$ git commit -m "Added login.sh file in login branch"
[login 9d1ca3b] Added login.sh file in login branch
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 login.sh
[ec2-user@ip-172-31-23-178 git_practice]$ git switch main
Switched to branch 'main'
[ec2-user@ip-172-31-23-178 git_practice]$ touch readme.md
[ec2-user@ip-172-31-23-178 git_practice]$ ls
README.md  readme.md
[ec2-user@ip-172-31-23-178 git_practice]$ vi README.md
[ec2-user@ip-172-31-23-178 git_practice]$ git switch login
M      README.md
Switched to branch 'login'
[ec2-user@ip-172-31-23-178 git_practice]$ git switch main
M      README.md
Switched to branch 'main'
[ec2-user@ip-172-31-23-178 git_practice]$ git add .
[ec2-user@ip-172-31-23-178 git_practice]$ git commit -m "Added a line README.md"
[main b5f75d6] Added a line README.md
 2 files changed, 1 insertion(+)
 create mode 100644 readme.md
[ec2-user@ip-172-31-23-178 git_practice]$ git switch login
Switched to branch 'login'
[ec2-user@ip-172-31-23-178 git_practice]$ vi README.md
[ec2-user@ip-172-31-23-178 git_practice]$ git add .
```

```

[ec2-user@ip-172-31-23-178 git_practice]$ git commit -m "Added different line in README.md file"
[login cd76ac3] Added different line in README.md file
1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-23-178 git_practice]$ git switch main
Switched to branch 'main'
[ec2-user@ip-172-31-23-178 git_practice]$ git merge login
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
[ec2-user@ip-172-31-23-178 git_practice]$ vi README.md
[ec2-user@ip-172-31-23-178 git_practice]$ git add .
[ec2-user@ip-172-31-23-178 git_practice]$ git commit -m "Resolved merge conflicts"
[main a15e7fc] Resolved merge conflicts
[ec2-user@ip-172-31-23-178 git_practice]$ ^C
[ec2-user@ip-172-31-23-178 git_practice]$ ^C
[ec2-user@ip-172-31-23-178 git_practice]$ git remote add origin https://github.com/nagaraj602/g
it_practice.git
[ec2-user@ip-172-31-23-178 git_practice]$ git push --set-upstream origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Compressing objects: 100% (9/9), done.
Writing objects: 100% (13/13), 1.23 KiB | 1.23 MiB/s, done.
Total 13 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/nagaraj602/git_practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
[ec2-user@ip-172-31-23-178 git_practice]$ 

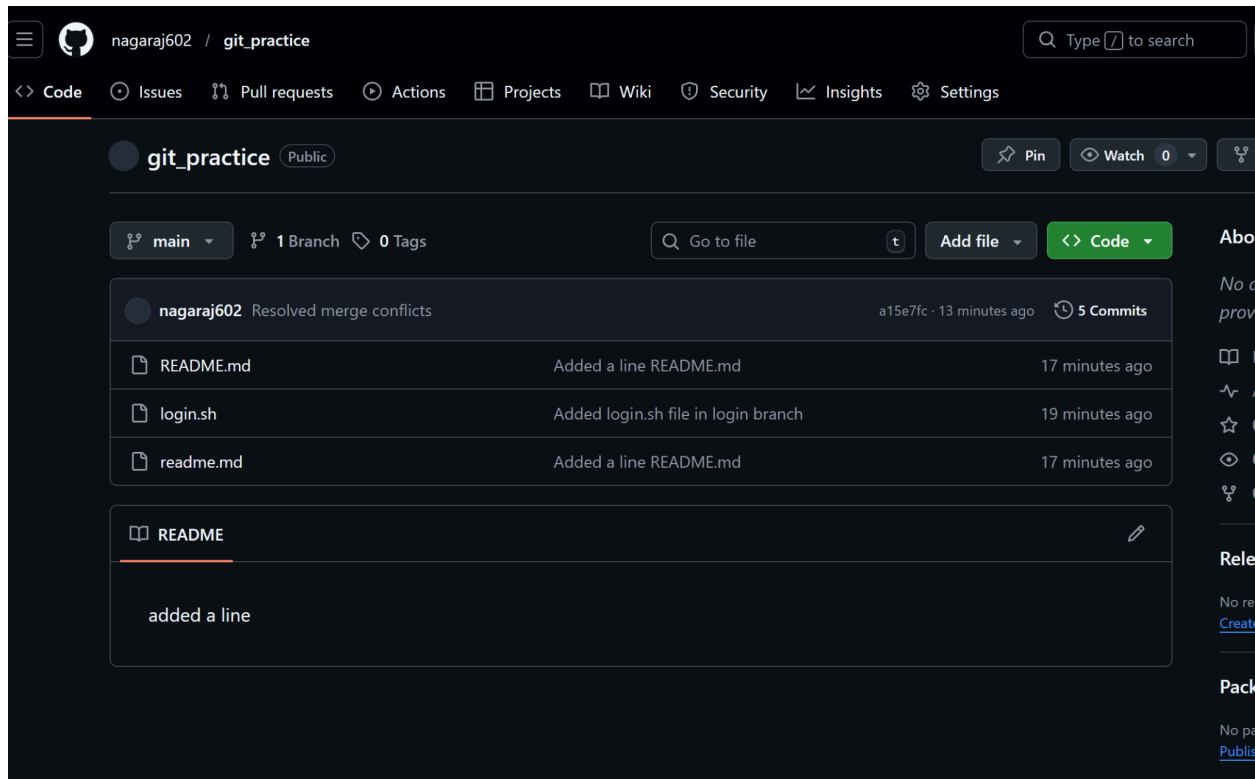
```



```

<<<<<<< HEAD
added a line
=====
Checking for merge conflict
>>>>>> login
~
~
~
~
~
~
~

```



6. Task2: Revert & Reset

- Create a Git repo `git_reset_revert_demo`.
- Add 3 commits by modifying a file `demo.txt`.
- Use `git log` to identify commit hashes.
- Use `git revert` to undo the second commit without removing history.
- Then use `git reset --hard` to go back to the first commit.
- Push the final repo to GitHub and share a screenshot of `git log --oneline`.

Ans:

→ `mkdir git_reset_revert_demo; cd git_reset_revert_demo`

→ `git init`

→ `touch demo.txt`

→ `git add .`

→ `git commit -m "Created file demo.txt"`

→ `vi demo.txt`

#Added first in it

→ `git add .`

→ `git commit -m "Added first line to the file demo.txt"`

→ `vi demo.txt`

#Added second line in it

→ `git add .`

→ `git commit -m "Added second line to the file demo.txt"`


```
→ git log
→ git log --oneline
o/p:
ed7551d (HEAD -> main) Added second line to the file demo.txt
1ca36cd Added first line to the file demo.txt
583e0b2 Created file demo.txt
```

Now we revert second commit:

```
→ git revert 1ca36cd
Auto-merging demo.txt
CONFLICT (content): Merge conflict in demo.txt
error: could not revert 1ca36cd... Added first line to the file demo.txt
hint: After resolving the conflicts, mark them with
hint: "git add/rm <paths>", then run
hint: "git revert --continue".
hint: You can instead skip this commit with "git revert --skip".
hint: To abort and get back to the state before "git revert",
hint: run "git revert --abort".
hint: Disable this message with "git config advice.mergeConflict false"
```

```
→ cat demo.txt
<<<<<<< HEAD
First line
Second line
=====
>>>>>>> parent of 1ca36cd (Added first line to the file demo.txt)
```

This means, you tried to revert the changes to the second commit, meaning, removing the 2nd commit "First line" will cause issues for your 3rd commit, as you made changes in the same file. Since 3rd commit depends on 2nd commit, git cannot revert it. We need to resolve conflicts manually. That is we need to remove "First line" and keep only "Second line" in demo.txt

```
→ vi demo.txt
After making changes in the file, it looks like this:
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ cat demo.txt
Second line
```

Now add the file to the staging area again.

```
→ git add .
```

Now continue the revert

```
→ git revert --continue
```

Go to insert mode and add the line as conflict resolved

:wq

Then the output shown on terminal will be:

```
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git revert --continue  
[main 332e277] Revert "Added first line to the file demo.txt"  
1 file changed, 1 deletion(-)
```

When we check the git log again, it shows new revert commit, by keeping commit history
→ git log --oneline

```
332e277 (HEAD -> main) Revert "Added first line to the file demo.txt"  
ed7551d Added second line to the file demo.txt  
1ca36cd Added first line to the file demo.txt  
583e0b2 Created file demo.txt
```

Now git reset --hard to go back to the first commit:

→ git reset --hard 583e0b2

```
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git reset --hard 583e0b2  
HEAD is now at 583e0b2 Created file demo.txt
```

→ git log --oneline

```
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git log --oneline  
583e0b2 (HEAD -> main) Created file demo.txt
```

→ git remote add origin https://github.com/nagaraj602/git_reset_revert_demo.git

→ git push --set-upstream origin main

```
[ec2-user@ip-172-31-23-178 ~]$ mkdir git_reset_revert_demo;cd git_reset_revert_demo
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git init
Initialized empty Git repository in /home/ec2-user/git_reset_revert_demo/.git/
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ touch demo.txt
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git add .
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git commit -m "Created file demo.txt"
[main (root-commit) 583e0b2] Created file demo.txt
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 demo.txt
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ vi demo.txt
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git add .
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git commit -m "Added first line to the file demo.txt"
[main 1ca36cd] Added first line to the file demo.txt
1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ vi demo.txt
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git add .
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git commit -m "Added second line to the file demo.txt"
[main ed7551d] Added second line to the file demo.txt
1 file changed, 1 insertion(+)
```

```
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git log
commit ed7551d83fae6c09cff140ee3abf3c7c8f21bac5 (HEAD -> main)
Author: nagaraj602 <nagarajkamath602@outlook.com>
Date: Fri Mar 6 09:32:20 2026 +0000

    Added second line to the file demo.txt

commit 1ca36cd993ab07d5af15701e4b141ff5183d40a3
Author: nagaraj602 <nagarajkamath602@outlook.com>
Date: Fri Mar 6 09:31:07 2026 +0000

    Added first line to the file demo.txt

commit 583e0b277803b50292c1163e2d3ea892a9ff22d0
Author: nagaraj602 <nagarajkamath602@outlook.com>
Date: Fri Mar 6 09:29:58 2026 +0000

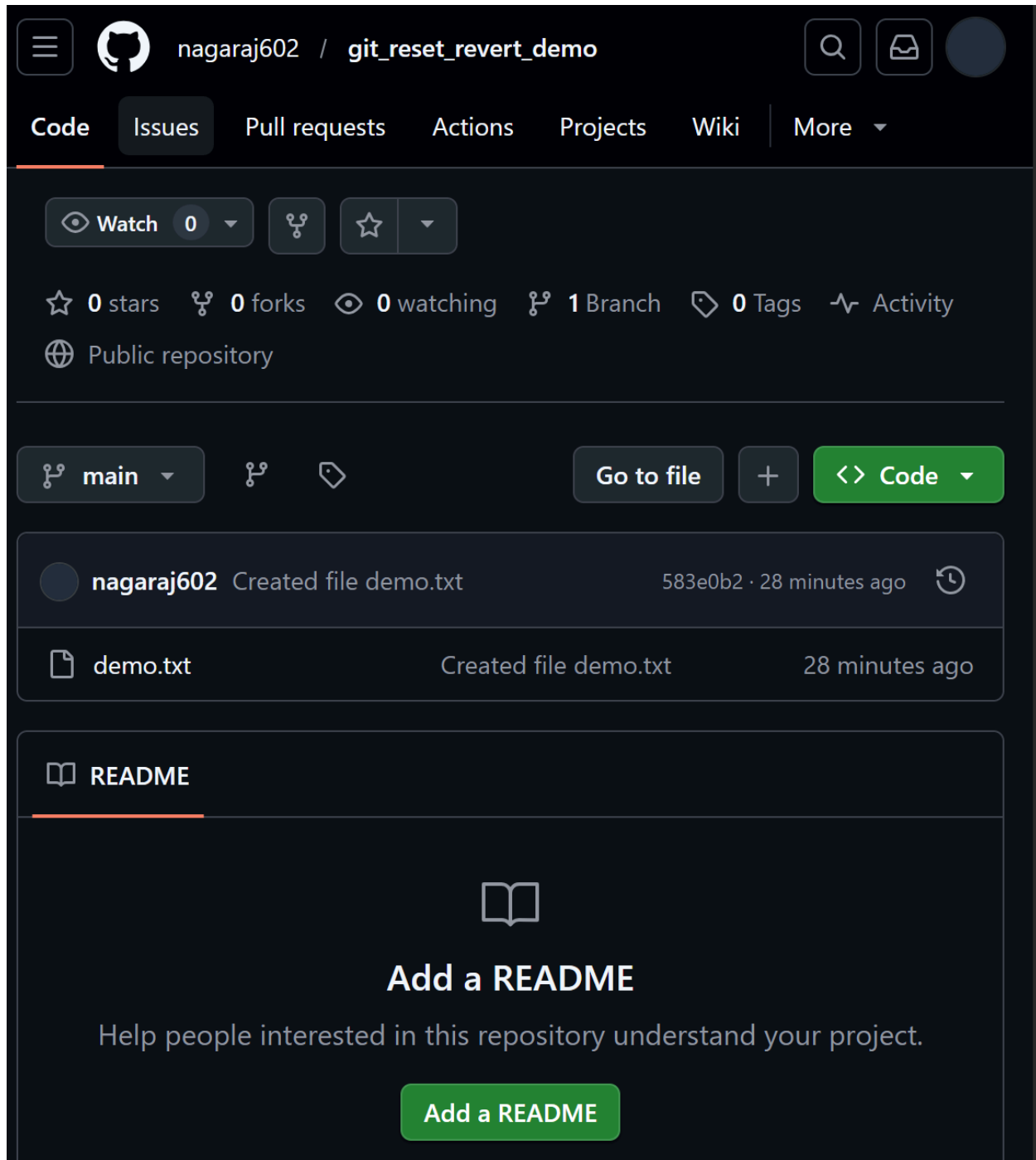
    Created file demo.txt
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$
```

```

[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git log --oneline
ed7551d (HEAD -> main) Added second line to the file demo.txt
1ca36cd Added first line to the file demo.txt
583e0b2 Created file demo.txt
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git revert 1ca36cd
Auto-merging demo.txt
CONFLICT (content): Merge conflict in demo.txt
error: could not revert 1ca36cd... Added first line to the file demo.txt
hint: After resolving the conflicts, mark them with
hint: "git add/rm <paths>", then run
hint: "git revert --continue".
hint: You can instead skip this commit with "git revert --skip".
hint: To abort and get back to the state before "git revert",
hint: run "git revert --abort".
hint: Disable this message with "git config advice.mergeConflict false"
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ vi demo.txt
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ cat demo.txt
<<<<<<< HEAD
First line
Second line
=====
>>>>>>> parent of 1ca36cd (Added first line to the file demo.txt)
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ vi demo.txt
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ cat demo.txt
Second line
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git add .
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git revert --continue
[main 332e277] Revert "Added first line to the file demo.txt"
1 file changed, 1 deletion(-)
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git log --oneline
332e277 (HEAD -> main) Revert "Added first line to the file demo.txt"
ed7551d Added second line to the file demo.txt
1ca36cd Added first line to the file demo.txt
583e0b2 Created file demo.txt
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git reset --hard 583e0b2
HEAD is now at 583e0b2 Created file demo.txt
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git log
commit 583e0b277803b50292c1163e2d3ea892a9ff22d0 (HEAD -> main)
Author: nagaraj602 <nagarajkamath602@outlook.com>
Date: Fri Mar 6 09:29:58 2026 +0000

    Created file demo.txt
[ec2-user@ip-172-31-23-178 git_reset_revert_demo]$ git log --oneline
583e0b2 (HEAD -> main) Created file demo.txt

```



7. Task3: Git Stash Scenario

- Create a repo `stash_practice`.
- Modify two files and don't commit them.
- Run `git stash` to temporarily save your work.
- Switch to a new branch `bugfix` and make a new commit.
- Return to `main` and apply your stash using `git stash apply`.
- Push final code and share output of `git stash list`.

Ans:

```
→ mkdir stash_practice
→ cd stash_practice
→ git init
→ touch stash.txt practice.txt
→ echo "Hello Stash" > stash.txt
→ echo "Hello Practice" > practice.txt
→ git add .
→ git commit -m "Initial commit"
→ echo "New change in stash file" >> stash.txt
→ echo "New change in practice file" >> practice.txt
→ git status
```

```
[ec2-user@ip-172-31-23-178 stash_practice]$ git status
```

On branch main

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: practice.txt

modified: stash.txt

no changes added to commit (use "git add" and/or "git commit -a")

```
→ git stash
```

```
[ec2-user@ip-172-31-23-178 stash_practice]$ git stash
```

Saved working directory and index state WIP on main: 5246f33 Initial commit

When we check git status again, working directory will be clean

```
→ git status
```

```
[ec2-user@ip-172-31-23-178 stash_practice]$ git status
```

On branch main

nothing to commit, working tree clean

```
→ git checkout -b bugfix
```

```
→ echo "Bugfix work" > bugfix.txt
```

```
→ git add .
```

```
→ git checkout main
```

```
→ git stash apply
```

```
[ec2-user@ip-172-31-23-178 stash_practice]$ git stash apply
```

On branch main

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: practice.txt

modified: stash.txt

no changes added to commit (use "git add" and/or "git commit -a")

- git add .
- git commit -m "Applied stash changes"
- git remote add origin https://github.com/nagaraj602/stash_practice.git
- git push --set-upstream origin main
- git push --set-upstream origin bugfix

```
[ec2-user@ip-172-31-23-178 ~]$ mkdir stash_practice;cd stash_practice
[ec2-user@ip-172-31-23-178 stash_practice]$ git init
Initialized empty Git repository in /home/ec2-user/stash_practice/.git/
[ec2-user@ip-172-31-23-178 stash_practice]$ touch stash.txt practice.txt
[ec2-user@ip-172-31-23-178 stash_practice]$ echo "Hello Stash" > stash.txt
[ec2-user@ip-172-31-23-178 stash_practice]$ echo "Hello Practice" > practice.txt
[ec2-user@ip-172-31-23-178 stash_practice]$ git add .
[ec2-user@ip-172-31-23-178 stash_practice]$ git stash
You do not have the initial commit yet
[ec2-user@ip-172-31-23-178 stash_practice]$ git commit -m "Initial commit"
[main (root-commit) 5246f33] Initial commit
 2 files changed, 2 insertions(+)
 create mode 100644 practice.txt
 create mode 100644 stash.txt
[ec2-user@ip-172-31-23-178 stash_practice]$ git status
On branch main
nothing to commit, working tree clean
[ec2-user@ip-172-31-23-178 stash_practice]$ echo "New change in stash file" >> stash.txt
echo "New change in practice file" >> practice.txt
[ec2-user@ip-172-31-23-178 stash_practice]$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   practice.txt
        modified:   stash.txt

no changes added to commit (use "git add" and/or "git commit -a")
[ec2-user@ip-172-31-23-178 stash_practice]$ git stash
Saved working directory and index state WIP on main: 5246f33 Initial commit
[ec2-user@ip-172-31-23-178 stash_practice]$ git status
On branch main
nothing to commit, working tree clean
[ec2-user@ip-172-31-23-178 stash_practice]$ git checkout -b bugfix
Switched to a new branch 'bugfix'
[ec2-user@ip-172-31-23-178 stash_practice]$ git add .
[ec2-user@ip-172-31-23-178 stash_practice]$ echo "Bugfix work" > bugfix.txt
[ec2-user@ip-172-31-23-178 stash_practice]$ git add .
[ec2-user@ip-172-31-23-178 stash_practice]$ git commit -m "Bugfix commit"
[bugfix dde2a0d] Bugfix commit
 1 file changed, 1 insertion(+)
 create mode 100644 bugfix.txt
[ec2-user@ip-172-31-23-178 stash_practice]$ git checkout main
Switched to branch 'main'
```

```

[ec2-user@ip-172-31-23-178 stash_practice]$ git stash apply
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   practice.txt
        modified:   stash.txt

no changes added to commit (use "git add" and/or "git commit -a")
[ec2-user@ip-172-31-23-178 stash_practice]$ git add .
git commit -m "Applied stash changes"
[main da6feb] Applied stash changes
2 files changed, 2 insertions(+)
[ec2-user@ip-172-31-23-178 stash_practice]$ git remote add origin https://github.com/nagaraj602/stash_practice.git
[ec2-user@ip-172-31-23-178 stash_practice]$ git push --set-upstream origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Compressing objects: 100% (4/4), done.
Writing objects: 100% (8/8), 622 bytes | 622.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/nagaraj602/stash_practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
[ec2-user@ip-172-31-23-178 stash_practice]$

```

The screenshot shows the GitHub interface for a repository named 'stash_practice' owned by 'nagaraj602'. The repository is public. At the top, there's a navigation bar with links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. Below this, the repository name 'stash_practice' is displayed with a 'Public' badge. There are buttons for 'Pin', 'Watch' (0), and a dropdown menu. A notification banner from 'bugfix' mentions recent pushes on Mar 6, with a 'Compare & pull request' button. Below the notification, the main branch is 'main', with 2 branches and 0 tags. A search bar 'Go to file' and buttons 'Add file' and '<> Code' are present. The commit history shows a single commit by 'nagaraj602' titled 'Applied stash changes' with hash 'da6feb' from 3 minutes ago, containing 2 commits. The commit details show two files: 'practice.txt' and 'stash.txt', both with the message 'Applied stash changes' and timestamp '3 minutes ago'. At the bottom, there's a 'README' section with a placeholder for a README file, including the text 'Add a README' and 'Help people interested in this repository understand your project.', with an 'Add a README' button.

8. Task4: Cherry-pick Practice

- Create a repo cherrypick_demo.
- Create a branch featureA and make 3 commits.

- Switch to main and cherry-pick only the 2nd commit from featureA.
- Verify using git log that only that commit was added.
- Push to GitHub and share screenshots of logs before & after cherry-pick.

Ans:

```
→ mkdir cherrypick_demo
→ cd cherrypick_demo
→ git init
→ echo "Initial file" > app.txt
→ git add .
→ git commit -m "Initial commit"

→ git checkout -b featureA
→ echo "Feature A - change 1" >> app.txt
→ git add .
→ git commit -m "FeatureA commit 1"

→ echo "Feature A - change 2" >> app.txt
→ git add .
→ git commit -m "FeatureA commit 2"

→ echo "Feature A - change 3" >> app.txt
→ git add .
→ git commit -m "FeatureA commit 3"

→ git log --oneline
```

```
45800c4 (HEAD -> featureA) FeatureA commit 3
8740090 FeatureA commit 2
821c584 FeatureA commit 1
c8a96b2 (main) Initial commit
```

```
→ git checkout main
→ git cherry-pick 8740090
It shows conflict:
```

```
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
error: could not apply 8740090... FeatureA commit 2
hint: After resolving the conflicts, mark them with
hint: "git add/rm <pathspec>", then run
hint: "git cherry-pick --continue".
hint: You can instead skip this commit with "git cherry-pick --skip".
hint: To abort and get back to the state before "git cherry-pick",
hint: run "git cherry-pick --abort".
```

hint: Disable this message with "git config advice.mergeConflict false"
[ec2-user@ip-172-31-23-178 cherrypick_demo]\$ vi app.txt
[ec2-user@ip-172-31-23-178 cherrypick_demo]\$ git cherry-pick 8740090
error: Cherry-picking is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: cherry-pick failed

→ vi app.txt

```
Initial file
<<<<<< HEAD
=====
Feature A - change 1
Feature A - change 2
>>>>>> 8740090
```

The conflict is because we are cherry picking the 2nd commit, but the 2nd commit depends on the 1st commit. So git cannot decide on merging. So conflict occurred. So we remove the first commit and the 3rd commit to keep only the 2nd commit.

→ vi app.txt

```
Initial file
Feature A - change 2
```

:wq

→ git add app.txt
→ git cherry-pick --continue

Added the line in the bottom of the file: "Cherry-pick conflict resolved"

:wq

→ git log --oneline

```
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git log --oneline
9b0a9d5 (HEAD -> main) FeatureA commit 2
c8a96b2 Initial commit
```

→ git remote add origin https://github.com/nagaraj602/cherrypick_demo.git
→ git push --set-upstream origin main
→ git push --set-upstream origin featureA

```

[ec2-user@ip-172-31-23-178 ~]$ mkdir cherrypick_demo
[ec2-user@ip-172-31-23-178 ~]$ cd cherrypick_demo
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git init
Initialized empty Git repository in /home/ec2-user/cherrypick_demo/.git/
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ echo "Initial file" > app.txt
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git add .
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git commit -m "Initial commit"
[main (root-commit) c8a96b2] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 app.txt
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git checkout -b featureA
Switched to a new branch 'featureA'
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ echo "Feature A - change 1" >> app.txt
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git add .
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git commit -m "FeatureA commit 1"
[featureA 821c584] FeatureA commit 1
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ echo "Feature A - change 2" >> app.txt
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git add .
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git commit -m "FeatureA commit 2"
[featureA 8740090] FeatureA commit 2
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ echo "Feature A - change 3" >> app.txt
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git add .
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git commit -m "FeatureA commit 3"
[featureA 45800c4] FeatureA commit 3
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git log --oneline
45800c4 (HEAD -> featureA) FeatureA commit 3
8740090 FeatureA commit 2
821c584 FeatureA commit 1
c8a96b2 (main) Initial commit
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git checkout main
Switched to branch 'main'
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git cherry-pick 8740090
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
error: could not apply 8740090... FeatureA commit 2
hint: After resolving the conflicts, mark them with
hint: "git add/rm <paths>", then run
hint: "git cherry-pick --continue".
hint: You can instead skip this commit with "git cherry-pick --skip".
hint: To abort and get back to the state before "git cherry-pick",
hint: run "git cherry-pick --abort".

```

```

hint: run "git cherry-pick --abort".
hint: Disable this message with "git config advice.mergeConflict false"
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ vi app.txt
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git cherry-pick 8740090
error: Cherry-picking is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: cherry-pick failed
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ vi app.txt
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ vi app.txt
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git add app.txt
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git cherry-pick --continue
[main 9b0a9d5] FeatureA commit 2
Date: Fri Mar 6 10:22:43 2026 +0000
1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ git log --oneline
9b0a9d5 (HEAD -> main) FeatureA commit 2
c8a96b2 Initial commit
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ 

```

9. Task5: Rebase Scenario

- Create a repo rebase_practice.
- Create branches main and feature/db.
- Add a few commits in both branches.
- Rebase feature/db on top of main.
- Resolve any conflicts and complete the rebase.
- Push both branches and share git log --graph --oneline --all.

Ans:

```

→ mkdir rebase_practice
→ cd rebase_practice
→ git init
→ echo "Initial code" > app.txt
→ git add .
→ git commit -m "Initial commit"

→ git checkout -b feature/db
→ echo "DB connection logic" >> app.txt
→ git add .
→ git commit -m "Add DB connection logic"

→ echo "DB query logic" >> app.txt
→ git add .
→ git commit -m "Add DB query logic"

→ git checkout main
→ echo "Main branch update" >> app.txt
→ git add .

```

→ git commit -m "Main branch update"

→ git checkout feature/db

→ git rebase main

Conflict occurred:

Auto-merging app.txt

CONFLICT (content): Merge conflict in app.txt

error: could not apply ee11c94... Add DB connection logic

hint: Resolve all conflicts manually, mark them as resolved with

hint: "git add/rm <conflicted_files>", then run "git rebase --continue".

hint: You can instead skip this commit: run "git rebase --skip".

hint: To abort and get back to the state before "git rebase", run "git rebase --abort".

hint: Disable this message with "git config advice.mergeConflict false"

Could not apply ee11c94... Add DB connection logic

→ cat app.txt

Initial code

<<<<<< HEAD

Main branch update

=====

DB connection logic

>>>>>> ee11c94 (Add DB connection logic)

→vi app.txt

Remove the conflict marker

→ cat app.txt

[ec2-user@ip-172-31-23-178 rebase_practice]\$ cat app.txt

Initial code

Main branch update

DB connection logic

→ git add app.txt

→ git rebase --continue

Go to bottom of the line >> add "Conflict cleared"

:wq

Now rebase continue command shows as:

[ec2-user@ip-172-31-23-178 rebase_practice]\$ git rebase --continue

[detached HEAD 7e479cf] Add DB connection logic

1 file changed, 1 insertion(+)

Successfully rebased and updated refs/heads/feature/db.

→ git log --graph --oneline --all

- * 6aa54d8 (HEAD -> feature/db, origin/feature/db) Add DB query logic
- * 7e479cf Add DB connection logic
- * 31cc795 (origin/main, main) Main branch update
- * 5c1ce49 Initial commit

→ git remote add origin https://github.com/nagaraj602/rebase_practice.git

→ git push --set-upstream origin main

→ git push --set-upstream origin feature/db

```
[ec2-user@ip-172-31-23-178 cherrypick_demo]$ cd
[ec2-user@ip-172-31-23-178 ~]$ mkdir rebase_practice
[ec2-user@ip-172-31-23-178 ~]$ cd rebase_practice
[ec2-user@ip-172-31-23-178 rebase_practice]$ git init
Initialized empty Git repository in /home/ec2-user/rebase_practice/.git/
[ec2-user@ip-172-31-23-178 rebase_practice]$ echo "Initial code" > app.txt
[ec2-user@ip-172-31-23-178 rebase_practice]$ git add .
[ec2-user@ip-172-31-23-178 rebase_practice]$ git commit -m "Initial commit"
[main (root-commit) 5c1ce49] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 app.txt
[ec2-user@ip-172-31-23-178 rebase_practice]$ git branch
* main
[ec2-user@ip-172-31-23-178 rebase_practice]$ git checkout -b feature/db
Switched to a new branch 'feature/db'
[ec2-user@ip-172-31-23-178 rebase_practice]$ git rebase main
Current branch feature/db is up to date.
[ec2-user@ip-172-31-23-178 rebase_practice]$ echo "DB connection logic" >> app.txt
[ec2-user@ip-172-31-23-178 rebase_practice]$ git add .
[ec2-user@ip-172-31-23-178 rebase_practice]$ git commit -m "Add DB connection logic"
[feature/db ee11c94] Add DB connection logic
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-23-178 rebase_practice]$ echo "DB query logic" >> app.txt
[ec2-user@ip-172-31-23-178 rebase_practice]$ git add .
[ec2-user@ip-172-31-23-178 rebase_practice]$ git commit -m "Add DB query logic"
[feature/db f556e21] Add DB query logic
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-23-178 rebase_practice]$ git checkout main
Switched to branch 'main'
[ec2-user@ip-172-31-23-178 rebase_practice]$ echo "Main branch update" >> app.txt
[ec2-user@ip-172-31-23-178 rebase_practice]$ git add .
[ec2-user@ip-172-31-23-178 rebase_practice]$ git commit -m "Main branch update"
[main 31cc795] Main branch update
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-23-178 rebase_practice]$ git checkout feature/db
Switched to branch 'feature/db'
[ec2-user@ip-172-31-23-178 rebase_practice]$ git rebase main
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
error: could not apply ee11c94... Add DB connection logic
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
```

```

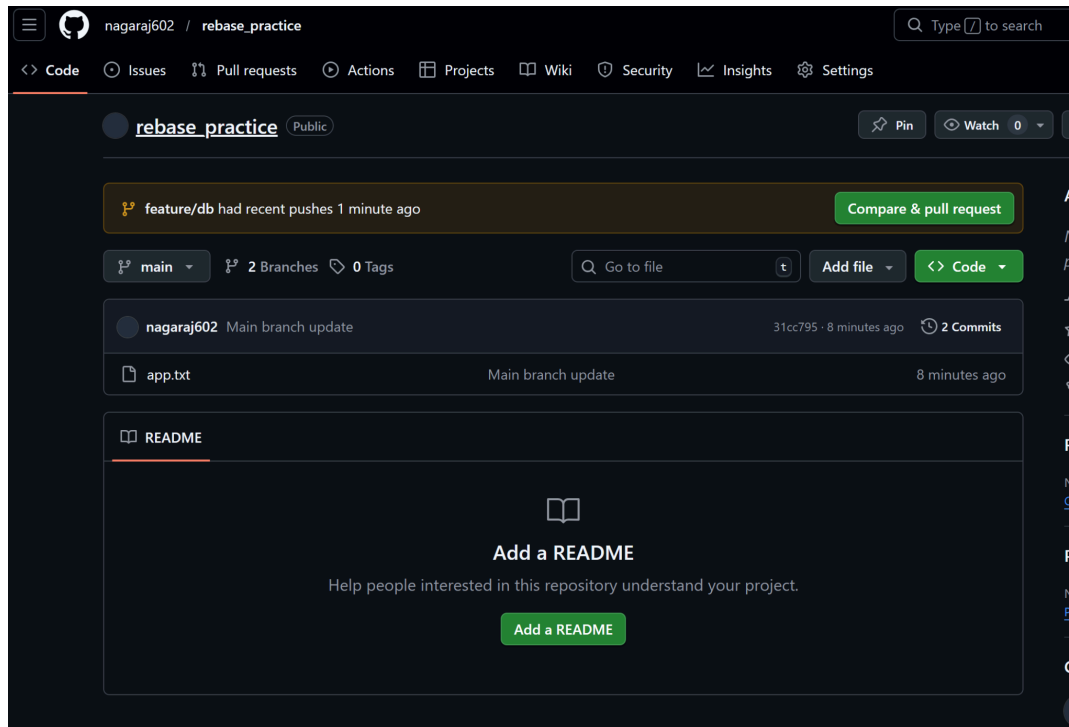
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config advice.mergeConflict false"
Could not apply ee11c94... Add DB connection logic
[ec2-user@ip-172-31-23-178 rebase_practice]$ vi app.txt
[ec2-user@ip-172-31-23-178 rebase_practice]$ cat app.txt
Initial code
Main branch update
DB connection logic
[ec2-user@ip-172-31-23-178 rebase_practice]$ git add app.txt
[ec2-user@ip-172-31-23-178 rebase_practice]$ git rebase --continue
[detached HEAD 7e479cf] Add DB connection logic
1 file changed, 1 insertion(+)
Successfully rebased and updated refs/heads/feature/db.
[ec2-user@ip-172-31-23-178 rebase_practice]$ git remote add origin https://github.com/nagaraj602/rebase_practice.git
[ec2-user@ip-172-31-23-178 rebase_practice]$ git push --set-upstream origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (6/6), 475 bytes | 475.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/nagaraj602/rebase_practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
[ec2-user@ip-172-31-23-178 rebase_practice]$ git push --set-upstream origin feature/db
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 554 bytes | 554.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
remote:
remote: Create a pull request for 'feature/db' on GitHub by visiting:
remote:   https://github.com/nagaraj602/rebase_practice/pull/new/feature/db
remote:
To https://github.com/nagaraj602/rebase_practice.git
 * [new branch]      feature/db -> feature/db
branch 'feature/db' set up to track 'origin/feature/db'.
[ec2-user@ip-172-31-23-178 rebase_practice]$ 
[ec2-user@ip-172-31-23-178 rebase_practice]$ 

```

```

[ec2-user@ip-172-31-23-178 rebase_practice]$ git log --graph --oneline --all
* 6aa54d8 (HEAD -> feature/db, origin/feature/db) Add DB query logic
* 7e479cf Add DB connection logic
* 31cc795 (origin/main, main) Main branch update
* 5c1ce49 Initial commit
[ec2-user@ip-172-31-23-178 rebase_practice]$ 

```



10. Task6: Tagging & Release

- Create a repo `release_versioning`.
- Make 3 commits (v1 feature, v2 feature, v3 feature).
- Tag each commit as v1.0, v2.0, v3.0.
- Push all tags to GitHub.
- Share output of `git tag -l` and screenshot from GitHub showing releases.

Ans:

```
→ mkdir release_versioning
→ cd release_versioning
→ git init
→ touch app.txt
→ echo "Version 1 feature" >> app.txt
→ git add .
→ git commit -m "v1 feature"
→ git tag v1.0
```

```
→ echo "Version 2 feature" >> app.txt
→ git add .
→ git commit -m "v2 feature"
→ git tag v2.0
```

```
→ echo "Version 3 feature" >> app.txt
→ git add .
→ git commit -m "v3 feature"
```


→ git tag v3.0

(OR) [instead of adding tag after each commit, we can list commits using git log --oneline and then add tag for required commit hash.

→ git log --oneline

1e0c222 (HEAD -> main, origin/main) v3 feature

debe2d8 v2 feature

0adb7b2 v1 feature

→ git tag v1.0 0adb7b2

→ git tag v2.0 debe2d8

→ git tag v3.0 1e0c222

]

List all the tags:

→ git tag -l

v1.0

v2.0

v3.0

→ git remote add origin https://github.com/nagaraj602/release_versioning.git

→ git push --set-upstream origin main

→ git push origin --tags

```

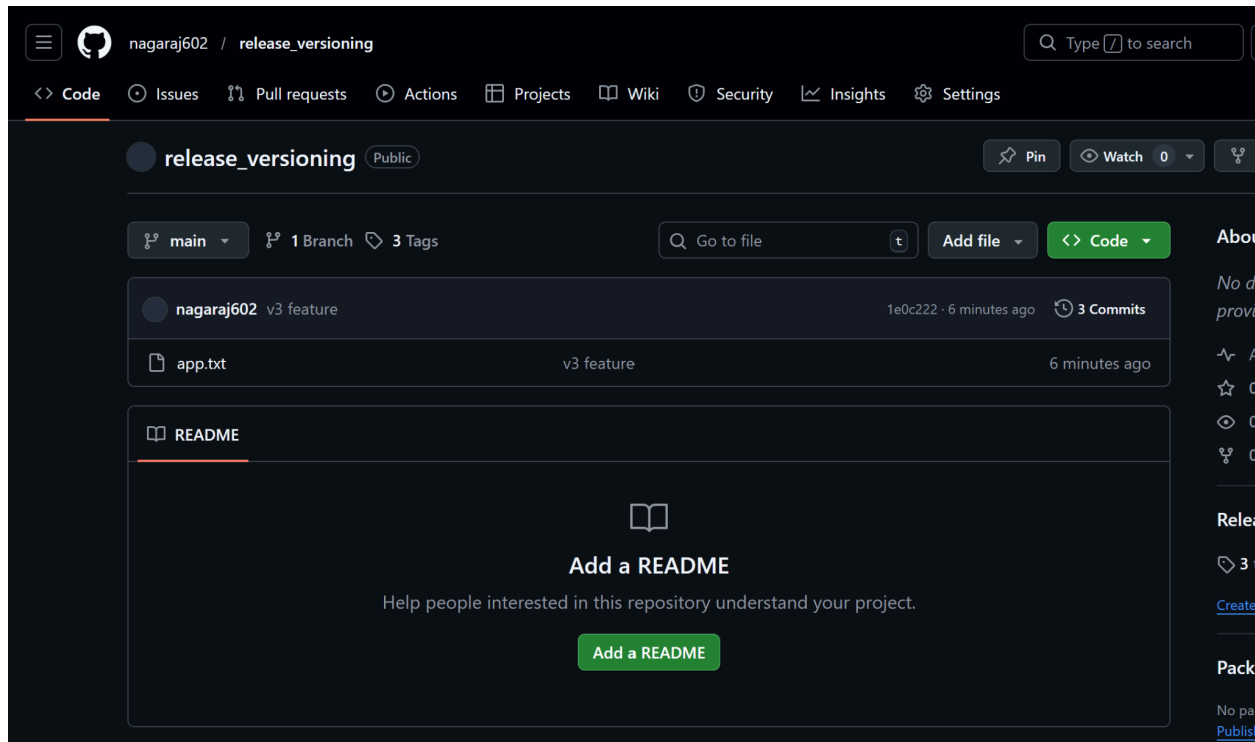
[ec2-user@ip-172-31-23-178 ~]$ mkdir release_versioning
[ec2-user@ip-172-31-23-178 ~]$ cd release_versioning
[ec2-user@ip-172-31-23-178 release_versioning]$ git init
Initialized empty Git repository in /home/ec2-user/release_versioning/.git/
[ec2-user@ip-172-31-23-178 release_versioning]$ touch app.txt
[ec2-user@ip-172-31-23-178 release_versioning]$ echo "Version 1 feature" >> app.txt
[ec2-user@ip-172-31-23-178 release_versioning]$ git add .
[ec2-user@ip-172-31-23-178 release_versioning]$ git commit -m "v1 feature"
[main (root-commit) 0adb7b2] v1 feature
1 file changed, 1 insertion(+)
create mode 100644 app.txt
[ec2-user@ip-172-31-23-178 release_versioning]$ git tag v1.0
[ec2-user@ip-172-31-23-178 release_versioning]$ echo "Version 2 feature" >> app.txt
[ec2-user@ip-172-31-23-178 release_versioning]$ git add .
[ec2-user@ip-172-31-23-178 release_versioning]$ git commit -m "v2 feature"
[main debe2d8] v2 feature
1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-23-178 release_versioning]$ git tag v2.0
[ec2-user@ip-172-31-23-178 release_versioning]$ echo "Version 3 feature" >> app.txt
[ec2-user@ip-172-31-23-178 release_versioning]$ git add .
[ec2-user@ip-172-31-23-178 release_versioning]$ git commit -m "v3 feature"
[main 1e0c222] v3 feature
1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-23-178 release_versioning]$ git tag v3.0
[ec2-user@ip-172-31-23-178 release_versioning]$ git tag -l
v1.0
v2.0
v3.0
[ec2-user@ip-172-31-23-178 release_versioning]$ git remote add origin https://github.com/nagaraj602/r
release_versioning.git
[ec2-user@ip-172-31-23-178 release_versioning]$ git push --set-upstream origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Compressing objects: 100% (4/4), done.
Writing objects: 100% (9/9), 695 bytes | 695.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/nagaraj602/release_versioning.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
[ec2-user@ip-172-31-23-178 release_versioning]$ git push origin --tags
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/nagaraj602/release_versioning.git
 * [new tag]         v1.0 -> v1.0
 * [new tag]         v2.0 -> v2.0

```

```

To https://github.com/nagaraj602/release_versioning.git
 * [new tag]         v1.0 -> v1.0
 * [new tag]         v2.0 -> v2.0
 * [new tag]         v3.0 -> v3.0
[ec2-user@ip-172-31-23-178 release_versioning]$ git log --oneline
1e0c222 (HEAD -> main, tag: v3.0, origin/main) v3 feature
debe2d8 (tag: v2.0) v2 feature
0adb7b2 (tag: v1.0) v1 feature
[ec2-user@ip-172-31-23-178 release_versioning]$ 

```



11. Task7: Git Collaboration Simulation

- Create a new repo on GitHub and clone it locally.
- Create a branch feature/api.
- Make some commits, push to GitHub, and open a Pull Request (PR).
- Merge the PR using GitHub UI.
- Pull changes back to your local repo and verify.
- Share link to the PR and output of git branch -a.

Ans:

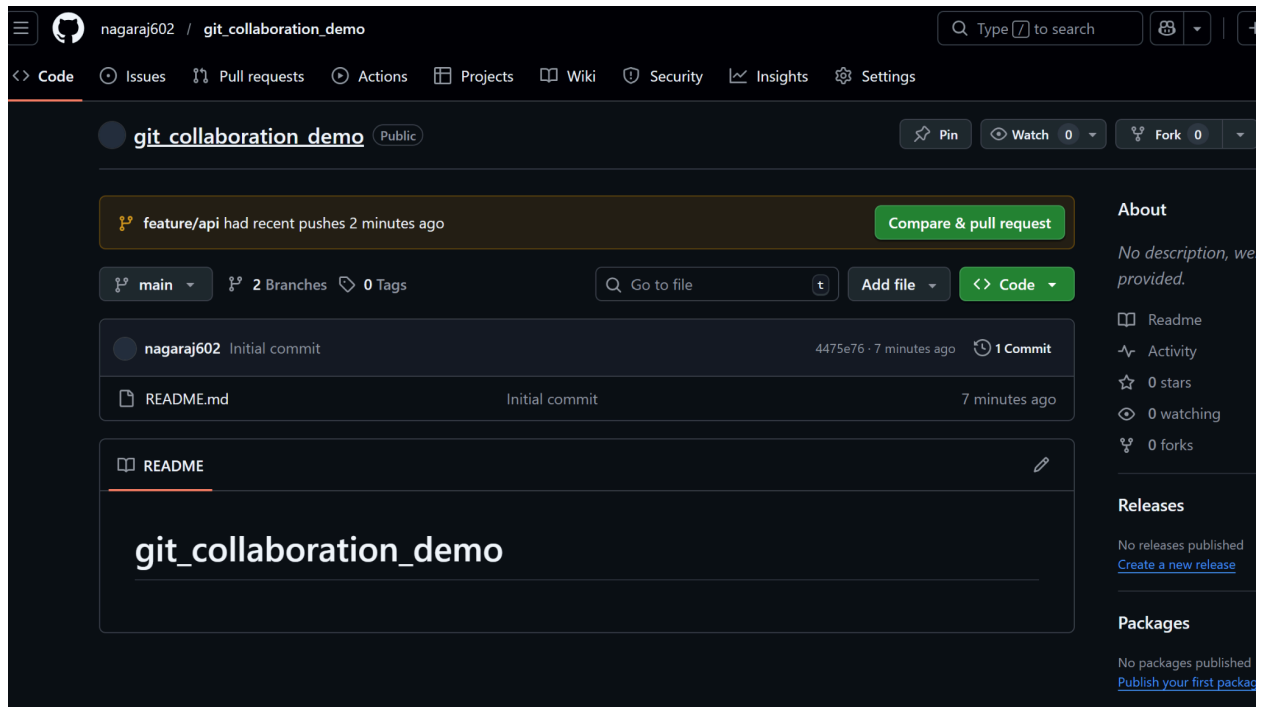
- Created a new repo in Github: "git_collaboration_demo" with the README.md file.
- Cloned the repo to local machine
- git clone https://github.com/nagaraj602/git_collaboration_demo.git
- cd git_collaboration_demo
- git checkout -b feature/api
- echo "API logic added" > api.txt
- git add .
- git commit -m "Added API feature"
- git push -u origin feature/api

Now go to github and it will show "Compare & Pull request" If not shown, we can create pull request.

Title: Added API feature

Description: Added initial API logic

- Create Pull Request
- If there is no conflict, then it gives option to "merge pull request"
- Click on "Merge Pull request"
- Add the commit message "Merge pull request #1 from nagaraj602/feature/api"
- Click "Confirm Merge"



Open a pull request

base: **main** ← compare: **feature/api** ✓ **Able to merge.** These branches can be automatically merged.

Add a title *

Added API feature

Add a description

Write Preview

H B I

Added initial API logic

Markdown is supported

Paste, drop, or click to add files

Create pull request

Reviewers

No reviews

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use [Closing keywords](#) in the description automatically close issues

Helpful resources

[GitHub Community Guidelines](#)

Added API feature #1

 Open nagaraj602 wants to merge 1 commit into `main` from `feature/api` 

Conversation 0 Commits 1 Checks 0 Files changed 1

nagaraj602 commented now

Added initial API logic

Added API feature

3fe6cd0

No conflicts with base branch
Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Add a comment

Write

Preview

H B I ≡ < > ↺ | ≡ ≡ ≡ | ✎ @ ↻ ↶ □

Add your comment here

Reviewers

No reviews

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

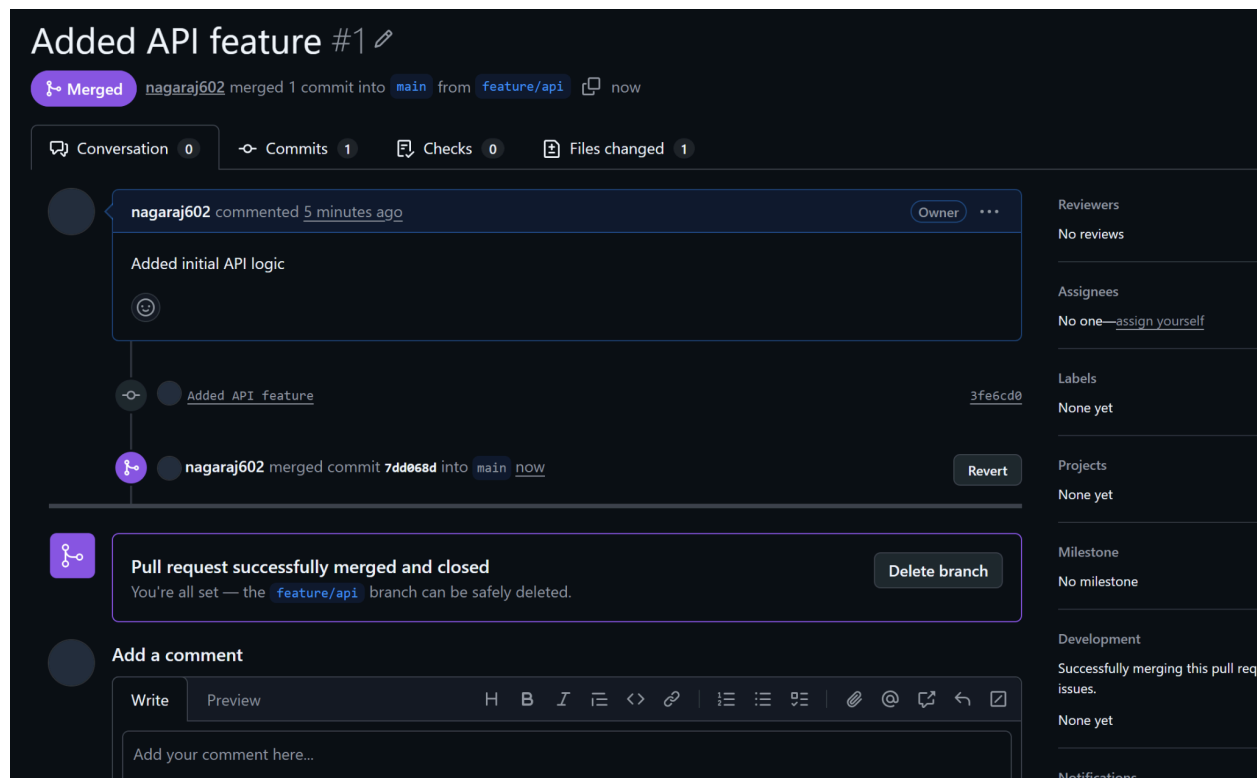
Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet



12. Create Organization in github and create a repo in it. Invite other members and create a branch for them and let each member upload their shell script assignment script to their respective branch.

Ans:

Create Organization: Click on your profile (Or click on Plus symbol which we use to create new repo, there it has new organization option) >> organization >> New organization >> Select Plan for organization: Free >> Enter Organization name: nagaraj-org >> contact email >> set as attach with personal account >> solve captcha and accept the terms and condition >> Invite the member by adding member username or email address or Full name >> It creates organization >> Invite other members if needed.

Now create new repo: shell_script >> create readme.md file >> Create branches for 3 members:

nagaraj
kumar
nithin

>> Go to your git >> clone the repo >>

git clone https://github.com/nagaraj-Org/shell_scripts.git >> git branch >> it shows only the main branch >> git clone won't add all the branches to the working directory, it adds to the remote tracking. So if you use → git branch -r or git branch -a → then it shows the remote tracking branches >> Now use: git switch nagaraj or git checkout nagaraj >> this brings your branch to your working directory. >> Now add all the shell

script files to your working directory >> add it to staging area using:
git add .
>> commit the changes:
git commit -m "Added Script files"
>> push the commit to remote repo from your nagaraj branch using:
git push --set-upstream origin nagaraj